PaisaPilot

AI-Powered Personal Finance Intelligence Platform

FULL-PHASE PRODUCT REQUIREMENTS DOCUMENT

India-first • SBI statement import • MongoDB • AI-driven financial insights

DOCUMENT DETAILS	DESCRIPTION
Product	PaisaPilot
Document type	Product Requirements Document (PRD)
Platform	Responsive web application
Initial data sources	User-uploaded SBI statements, CSV files, manual entries and synthetic demo data
Primary stack	Next.js, React, TypeScript, Tailwind CSS, Node.js, MongoDB Atlas
Delivery strategy	Phased MVP → AI insights → forecasting → eligible bank-data integration

Prepared as a practical product and engineering blueprint. Provider integrations and regulatory eligibility must be verified before production use.

1. Product Overview

Product vision: Help people understand where their money goes, identify spending patterns, anticipate potential cash-flow problems, and make informed decisions using secure transaction analysis and AI.

PaisaPilot is a personal finance intelligence platform for Indian users. Users can import SBI bank statements, categorize transactions, monitor expenses, set budgets, identify recurring payments, and ask an AI assistant to explain their finances using verified transaction data.

Problem statement

Financial records are spread across UPI applications, bank accounts, cards, cash payments and subscriptions. Transaction histories often do not provide a unified, actionable view. Users may overlook recurring charges, underestimate small purchases, miss spending changes and struggle to plan savings.

Product objectives

- Consolidate imported and manually entered transactions into one dashboard.
- Reduce manual expense categorization and identify recurring payments.
- Explain monthly spending patterns in plain language.
- Support budgets, alerts and transparent cash-flow projections.
- Protect sensitive financial information with secure architecture.
- Prepare a foundation for future authorized bank-data integrations.

Target users

Segment	Primary need
College students	Manage allowances and everyday spending
Young professionals	Balance salary, bills, savings and lifestyle
Freelancers	Plan around irregular income
Families	Understand household spending
Small-business owners (later)	Explore cash-flow insights

2. Recommended Technology Stack

Layer	Recommendation	Notes
Frontend	Next.js App Router, React, TypeScript, Tailwind CSS	Responsive web UI
UI and charts	shadcn/ui, Lucide, Recharts	Accessible components and analytics
Backend	Next.js Route Handlers initially	Modular services; add Express only if justified
Database	MongoDB Atlas + MongoDB driver or Mongoose	Schema validation, indexes and scoped queries
Authentication	Auth.js or established auth provider	Secure sessions and account lifecycle
Validation	Zod	Validate API input and imported rows
File storage	Private object storage	Temporary statement files with deletion policies
Background jobs	Durable queue + Node.js worker	Parsing, categorization and scheduled alerts
AI	LLM API with structured output and tool calling	Grounded explanations, restricted tools
Testing	Vitest, React Testing Library, Playwright	Unit, integration and end-to-end tests
Monitoring	Sentry + structured logs	Errors, latency and worker health
Deployment	Vercel + managed worker hosting + MongoDB Atlas	Separate web requests from long jobs

Backend architecture decision

Start with Next.js Route Handlers and a separate background worker. Use the API for authenticated CRUD, analytics and AI chat. Use workers for statement parsing, bulk imports, recurring analysis and long-running jobs. Introduce a separate Express API when independent scaling, multiple clients or team structure makes it worthwhile.

Suggested supporting tools

- GitHub Actions for CI, automated checks and deployment workflows.
- A managed email service for verification and notifications.
- A rate-limiting solution suitable for the chosen hosting environment.
- Private object storage with lifecycle expiration for uploaded statements.
- An error-monitoring and performance-monitoring service.

3. Functional Requirements by Phase

Phase 1 — Foundation and authentication

- Email/password registration, email verification, login and password reset.
- Optional Google sign-in, secure sessions and protected routes.
- User profile, preferred currency and timezone.
- Privacy notice, data export and deletion request flows.
- Server-side authorization on every private endpoint.

Acceptance criteria: Unauthenticated users cannot access financial data; private queries are scoped to the authenticated user; reset tokens expire and cannot be reused; deletion follows the documented retention

policy.

Phase 2 — Transaction import and management

Inputs: CSV statements, supported SBI PDF statements, manual entries, synthetic demo data, and future bank formats through separate parsers.

- Upload progress, import status, preview and column mapping.
- Validate file type, size, row structure, dates and amounts.
- Normalize transactions and preserve original descriptions.
- Detect duplicates without relying solely on date and amount.
- Report rejected rows, import errors and retry status.
- Support search, filters, sorting, pagination and import history.

Import workflow: Upload → validate → parse → preview → confirm → deduplicate → save → refresh analytics.

Phase 3 — Dashboard and analytics

- Income, expenses, net cash flow and category totals for selected periods.
- Monthly trends, recent transactions, budgets and recurring-payment summaries.
- Date-range filters, equivalent-period comparisons and drill-down to transactions.
- Clear loading, empty, error and partial-data states.
- Distinguish imported transaction summaries from live bank balances.

Phase 4 — AI expense categorization

- Apply deterministic merchant mappings and rules before AI classification.
- Validate predictions against an allowed category list.
- Record method, confidence/review status and user corrections.
- Allow custom categories and recategorization.
- Evaluate performance against manually verified examples.

Phase 5 — AI financial assistant

Example user questions include: “How much did I spend on food last month?”, “Why are my expenses higher?”, “Which recurring payments should I review?” and “What happens if I spend another ₹2,000 this week?”

Expose approved backend tools such as getSpendingSummary(period), getSpendingByCategory(period, category), comparePeriods(periodA, periodB), getBudgetStatus(), getRecurringPayments() and simulateCashFlow(scenario).

- AI must not directly query MongoDB or generate arbitrary database queries.
- Backend functions validate identity, permissions, dates and parameters.
- The backend calculates financial values; the model explains verified results.
- Treat transaction descriptions and uploaded documents as untrusted input.
- Do not permit transfers, payments or other financial actions in the assistant.

Phase 6 — Budgets, recurring payments and alerts

- Weekly and monthly budgets, category limits and utilization.
- Configurable warning thresholds and remaining-budget display.
- Recurring-payment candidates based on merchant, amount and interval patterns.
- User confirmation for uncertain recurring-payment detections.

- Alerts for budget limits, unusual spending, possible duplicates and upcoming bills.

Phase 7 — Cash-flow forecasting

- Use historical transactions, confirmed bills and user-entered expected income.
- Show projected cash flow, possible shortfall dates and scenario comparisons.
- Begin with transparent deterministic projections; add statistical models when enough reliable history exists.
- Show data coverage, assumptions and uncertainty; never guarantee future balances.

Phase 8 — Authorized bank-data synchronization

- Evaluate eligible Account Aggregator or other authorized provider integrations.
- Implement consent, purpose limitation, revocation and provider callbacks.
- Track synchronization status, last refresh and failures.
- Normalize and deduplicate synchronized records through the same transaction pipeline.
- Verify provider coverage, business eligibility, commercial terms and applicable compliance requirements before implementation.

4. MongoDB Data Model

Collection	Purpose
users	Profile, preferences and account status
accounts	User-defined financial accounts and connection metadata
transactions	Normalized transaction records
imports	Statement processing jobs and import history
budgets	Category and period-based spending limits
categories	System and user-defined categories
recurringPayments	Suspected and confirmed recurring charges
insights	Generated explanations and evidence references
conversations / messages	AI conversation history with retention controls
consents	Provider access, consent status and permitted purposes
notifications	Alerts and delivery status
auditEvents	Security-sensitive application events

Transaction document fields

```
userId, accountId, source, transactionDate, description, merchant, amountMinor, currency, direction,
categoryId, subcategoryId, importId, deduplicationKey, categorizationStatus, createdAt, updatedAt
```

Store INR amounts in integer paise where practical and use an explicit sign/direction convention. Preserve original descriptions and provenance. Avoid embedding unlimited transactions or chat history in one user document.

Suggested indexes

```
db.transactions.createIndex({ userId: 1, transactionDate: -1, _id: -1 }); db.transactions.createIndex({ userId:
1, categoryId: 1, transactionDate: -1 }); db.imports.createIndex({ userId: 1, createdAt: -1 });
db.budgets.createIndex({ userId: 1, periodStart: 1, periodEnd: 1 });
```

Use a carefully designed unique key scoped to the user and source account for deduplication. Legitimate purchases may share the same amount and date, so those fields alone are not a safe duplicate key. Add

schema validation and test index performance against real query patterns.

5. High-Level Architecture

Component	Responsibility
Next.js web client	Dashboard, transaction management, budgets, chat and settings
Application API	Authentication, authorization, validation, CRUD and analytics
MongoDB Atlas	Users, transactions, budgets, import metadata and insights
Private object storage	Temporary statement files with lifecycle deletion
Job queue and workers	Parsing, categorization, recurring detection and scheduled alerts
AI service	Classification and grounded natural-language explanations
Notification service	Email and in-app alert delivery

Scalability principles

- Use asynchronous jobs for expensive parsing and AI processing.
- Make imports idempotent so retries do not create duplicate records.
- Use indexes, pagination and query profiling.
- Cache selected aggregate results only where appropriate and define invalidation.
- Keep AI calls outside critical financial write operations.
- Scale workers separately before splitting the backend into microservices.
- Maintain monitoring, backups and tested recovery procedures.

6. Security and Privacy Requirements

- Authentication with secure session management and server-side authorization.
- Strict user-level data isolation for every database query.
- TLS in transit and encryption at rest.
- Secure secret management and API-key rotation.
- File validation, upload size limits and private storage.
- Rate limiting, secure cookies and CSRF protection where applicable.
- No sensitive transaction details, bank credentials or tokens in logs.
- Audit events for security-sensitive actions.
- Data export, deletion, retention and backup recovery controls.
- Prompt-injection defenses and restricted AI tool access.
- Dependency scanning and automated security tests.

Never request users' UPI PINs, banking passwords or OTPs. Minimize financial data sent to external AI providers. Before handling real financial data, review applicable Indian privacy and financial-sector requirements with qualified professionals. For live integrations, verify provider and regulatory requirements.

7. Non-Functional Requirements

Area	Proposed requirement
Performance	Target p95 below 500 ms for ordinary indexed API reads, excluding AI and file processing
Reliability	Durable jobs, retry policies, idempotency and graceful failure
Availability	Define an uptime target suitable for the deployment tier
Scalability	Horizontal worker scaling and indexed transaction queries
Accessibility	Responsive layout, keyboard navigation and readable charts
Compatibility	Modern desktop and mobile browsers
Data integrity	Accurate calculations and validated import records
Observability	Request tracing, error monitoring, queue metrics and structured logs
Privacy	Minimal collection, documented retention and user-controlled deletion
AI quality	Evaluation dataset, grounded answers and visible uncertainty

8. API Requirements

Method	Endpoint	Purpose
GET	/api/transactions	List and filter transactions
POST	/api/transactions	Create a manual transaction
PATCH	/api/transactions/:id	Edit an authorized transaction
DELETE	/api/transactions/:id	Delete a transaction
POST	/api/imports	Initiate statement import
GET	/api/imports/:id	Check import status
POST	/api/imports/:id/confirm	Confirm a previewed import
GET	/api/analytics/summary	Retrieve calculated summaries
GET	/api/analytics/categories	Retrieve category breakdown
GET/POST	/api/budgets	List or create budgets
GET	/api/insights	List generated insights
POST	/api/assistant/chat	Ask a finance question
GET	/api/recurring-payments	Retrieve recurring-payment candidates

All endpoints must enforce authentication, authorization, input validation, pagination, consistent response schemas and safe error handling. Do not expose database internals or stack traces to clients.

9. Release Roadmap

Phase	Scope	Deliverable
1. Foundation	Authentication, app shell, database, security controls	Secure application skeleton
2. MVP	Statement import, transactions, categorization, charts	Usable finance tracker
3. Intelligence	AI assistant, budgets, recurring payments, alerts	AI-powered finance assistant
4. Prediction	Scenario simulator, cash-flow forecast and shortfall alerts	Proactive planning features
5. Production	Security review, tests, monitoring, backups, accessibility	Production-ready release candidate
6. Integration	Provider selection, consent lifecycle and sync reconciliation	Eligible live-data integration

These are release milestones rather than guaranteed timelines. Live bank synchronization should not begin until provider access and compliance requirements have been confirmed.

10. Suggested Project Structure

```
paisapilot/ src/ app/ (auth)/login/ and signup/ (dashboard)/overview/ (dashboard)/transactions/
(dashboard)/budgets/ (dashboard)/recurring/ (dashboard)/insights/ (dashboard)/assistant/ (dashboard)/settings/
api/transactions/ api/imports/ api/budgets/ api/insights/ api/assistant/ components/ ui/ charts/ dashboard/
transactions/ modules/ auth/ transactions/ imports/ categorization/ analytics/ budgets/ forecasting/ ai/ lib/
mongodb/ auth/ validation/ security/ observability/ types/ workers/ statement-import/ categorization/
notifications/ tests/ unit/ integration/ e2e/ public/ .env.example package.json
```

Keep route handlers thin and put financial logic in reusable service modules. Treat the structure as a starting point; adapt it as the application grows.

11. MVP Definition of Done

- A user can register, log in and manage their account securely.
- A user can import a supported SBI statement or CSV file.
- Transactions are previewed and validated before committing.
- Repeated imports do not create unintended duplicates.
- Users can search, filter, categorize and correct transactions.
- Dashboard totals match independently verified calculations.
- Users can view category breakdowns and monthly comparisons.
- The AI assistant answers using verified backend results.
- A user cannot read or modify another user's data.
- File processing, retries and errors are handled safely.
- Users can export and delete their data.
- Automated tests cover financial calculations, authorization and imports.
- The app is deployed and tested on desktop and mobile browsers.

12. Final Recommendations

Recommended initial stack: Next.js App Router, React, TypeScript, Tailwind CSS and shadcn/ui; Next.js Route Handlers with modular services; MongoDB Atlas; established authentication; Zod; private object storage; a durable queue and Node.js workers; an LLM API with structured outputs; Vitest and Playwright; Vercel for the web application; and managed infrastructure for workers and MongoDB.

Keep the first release focused on secure authentication, reliable transaction imports, correct financial calculations, useful analytics and grounded AI explanations. Add forecasting after the data pipeline is reliable, and pursue authorized bank-data integration only after confirming eligibility.

Implementation references

- Next.js documentation: https://nextjs.org/docs
- MongoDB documentation: https://www.mongodb.com/docs/
- OWASP File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- Use current official provider documentation for authentication, AI APIs, deployment and any regulated financial-data integration.